

Bancos de Dados Não-Relacionais

Aula 01: O Paradigma Pós-Relacional & Diretrizes da Disciplina

A presentation by

Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

Agenda do Encontro

Diretrizes & Avaliação

Contexto Histórico & Big Data

Gargalos do Relacional

O Movimento NoSQL & Modelos

Prática & Conclusão

Diretrizes & Avaliação

Visão Geral da Disciplina: ADS16

Estrutura do Curso

- **Carga Horária:** 40 horas no semestre.
- **Total de Encontros:** 20 encontros (2h/aula cada).
- **Horário Regular:** Terças-feiras.
- **Ambiente:** Laboratório de Informática.
- **Metodologia:** Aulas teórico-práticas com programação.

Pilares de Aprendizagem

- **Ambiente Docker:** Orquestração rápida de Redis, MongoDB e Neo4j.
- **Engenharia de Dados:** Alta escala, concorrência e particionamento.
- **Modernização:** Introdução a Bancos Vetoriais para Inteligência Artificial.

Sistema de Avaliação da Disciplina

Fórmula Regimental de Média por Período (N1 e N2):

$$\text{Nota} = (\text{AVT} \times 0,40) + (\text{AVP} \times 0,40) + (\text{Listas} \times 0,20)$$

AVT (40%) – Avaliação Teórica

- Prova individual sem consulta.
- Fundamentos, CAP/PACELC, ACID vs BASE e arquitetura.

AVP (40%) – Avaliação Prática

- **AVP1 (N1):** Desafio prático individual em laboratório.
- **AVP2 (N2):** Projeto prático integrado em **duplas**.

Listas (20%) – Avaliação Contínua

- Exercícios práticos de laboratório e fixação.
- Entrega até a data da avaliação teórica do período.

Avaliação Final (AF)

- Prova final de recuperação ao término do semestre.

Diretriz dos Sábados Letivos

10/10/2026 – Sábado Letivo 01

- **Dia Sem Conteúdo Novo.**
- **Oficina Prática de Ferramentas:** Orquestração de containers com Docker Compose.
- Subida e teste de instâncias de Redis e MongoDB.
- Resolução orientada da Lista Preparatória 1.

14/11/2026 – Sábado Letivo 02

- **Dia Sem Conteúdo Novo.**
- **Laboratório de Fixação:** Simulado e revisão prática integrada (Redis + MongoDB).
- Plantão de dúvidas antes de AVP1 (17/11) e AVT1 (24/11).
- Fechamento da Lista de Exercícios 1.

Contexto Histórico & Big Data

O Domínio do Modelo Relacional (1970 – 2000)

A Era Codd e a Álgebra Relacional

- Proposto por Edgar F. Codd em 1970 (IBM).
- Base matemática na álgebra relacional e conjuntos.
- Tabelas rígidas com chaves primárias e integridade.
- **Premissa Central:** Modelo desenhado para operar em **um único servidor centralizado**.

Garantias Transacionais ACID

- **Atomicidade:** Tudo ou nada na transação.
- **Consistência:** Regras e restrições sempre válidas.
- **Isolamento:** Concorrência sem interferência.
- **Durabilidade:** Persistência garantida em disco.
- **Sucesso:** Eliminou inconsistências por 30 anos.

O Ponto de Inflexão: A Web 2.0 e os 3 Vs

Volume & Velocidade

- **Volume:** Salto de gigabytes para terabytes e petabytes diários gerados por bilhões de usuários.
- Volume total excede o hardware de uma única máquina.
- **Velocidade:** Milhões de requisições por segundo.
- Exigência de tempo de resposta em milissegundos.

Variedade

- Dados heterogêneos: JSON, logs, redes e sensores IoT.
- Esquemas que mudam diariamente sem possibilidade de paradas programadas para migração (`ALTER TABLE`).

Gargalos do Relacional

Escala Vertical vs. Escala Horizontal

Escala Vertical (Scale-Up)

A abordagem clássica dos RDBMS:

- Expandir CPU, memória RAM e SSD do servidor principal.
- **Custo Exponencial:** Máquinas de topo custam valores astronômicos.
- **Limite Físico:** Teto de barramento e processamento.
- **Ponto Único de Falha:** Queda do servidor paralisa o sistema.

Escala Horizontal (Scale-Out)

A abordagem nativa do NoSQL:

- Adicionar servidores comuns em cluster (*commodity hardware*).
- **Custo Linear:** Expansão elástica sob demanda na nuvem.
- **Alta Resiliência:** Queda de nós não derruba o cluster.
- **Escala Elástica:** Novos nós dividem a carga.

O Custo dos JOINS em Clusters

Execução Local vs Distribuída

- Em uma máquina, um JOIN busca ponteiros em memória RAM ou disco local.
- Em um cluster particionado, tabelas diferentes residem em servidores físicos distintos.

O Gargalo do Network Shuffle

- Cruzar dados exige tráfego massivo de rede entre múltiplos servidores.
- **Consequência:** A latência salta de milissegundos para dezenas de segundos!

Descasamento de Impedância Objeto-Relacional

A Visão da Aplicação (OO)

- Código em Python trabalha com grafos ricos de objetos em memória.
- Objeto `Cliente` contém listas de `pedidos` e objetos de `endereço`.

A Fricção do Mapeamento (ORM)

- Para salvar, o ORM precisa fatiar o objeto em 4 tabelas relacionais.
- Para ler, exige múltiplos `JOINS` para remontar o estado em memória.

A Solução em Documentos NoSQL

- Documento JSON/BSON armazenado **exatamente** no formato do objeto.
- Gravação e leitura em **um único I/O atômico de disco**.
- Elimina camadas complexas e lentas de tradução objeto-relacional.

O Movimento NoSQL & Modelos

O Conceito: Not Only SQL

Origem e Filosofia

- Movimento consolidado em 2009 em São Francisco.
- Não significa rejeitar o SQL, mas sim **Not Only SQL** (Não Apenas SQL).
- **Premissa Fundamental:** Nem todos os dados do mundo se comportam como tabelas relacionais estritas.
- Foco em escalabilidade horizontal nativa, esquemas flexíveis (schemaless) e alta disponibilidade.

Persistência Poliglota

- Sistemas modernos utilizam bancos relacionais e NoSQL juntos.
- Escolhe-se a tecnologia mais adequada para cada domínio.
- **Exemplo:** PostgreSQL (pagamentos) + Redis (cache de sessões) + MongoDB (catálogo flexível).

Modelos NoSQL: Chave-Valor & Documentos

1. Chave-Valor (Key-Value)

Exemplos: Redis, DynamoDB, Memcached.

- Mapeamento direto `chave` → `valor` em memória.
- Altíssima taxa de IOPS e latência sub-milissegundo.
- **Casos de Uso:** Cache acelerador, controle de sessão de login e contadores em tempo real.

2. Documentos (Document Store)

Exemplos: MongoDB, Couchbase, Firestore.

- Dados armazenados em documentos JSON/BSON aninhados.
- Suporte nativo a arrays, objetos embutidos e índices.
- **Casos de Uso:** Catálogos de e-commerce, perfis de usuários e cadastros com esquema evolutivo.

Modelos NoSQL: Colunas & Grafos

3. Família de Colunas (Wide-Column)

Exemplos: Apache Cassandra, ScyllaDB, HBase.

- Linhas com colunas dinâmicas em anel distribuído.
- Otimizado para altíssimo volume de gravação contínua.
- **Casos de Uso:** Telemetria IoT, séries temporais, métricas de monitoramento e logs analíticos.

4. Grafos (Graph Database)

Exemplos: Neo4j, Amazon Neptune, ArangoDB.

- Vértices (nós), arestas (relacionamentos) e propriedades.
- Travessia direta de ponteiros sem múltiplos JOINS.
- **Casos de Uso:** Redes sociais, motores de recomendação e detecção de fraudes financeiras.

Bancos Vetoriais & IA (Modernização)

Vector Databases na Era da IA

- Dados não estruturados convertidos em vetores matemáticos multidimensionais (**embeddings**).
- Busca por **proximidade semântica** (similaridade por cosseno), não por correspondência de texto exato.
- **Exemplos:** ChromaDB, Pinecone, Qdrant, Milvus.

Aplicações Críticas: RAG

- **RAG (Retrieval-Augmented Generation):** Recuperação semântica para abastecer Modelos de Linguagem (LLMs).
- **Tópico Modernizado:** Nossa disciplina contará com uma aula prática dedicada (Aula 16) sobre este modelo!

Prática & Conclusão

Demonstração Prática: JSON vs 4 Tabelas Relacionais

Relacional: 4 Tabelas + 3 JOINS

```
SELECT c.nome, e.cidade, p.  
       id_pedido,  
       i.produto, i.quantidade  
FROM   clientes c  
JOIN   enderecos e ON c.id = e.  
       id_cliente  
JOIN   pedidos p ON c.id = p.  
       id_cliente  
JOIN   itens_pedido i  
       ON p.id_pedido = i.id_pedido  
WHERE  c.id = 1;
```

Exige remontar objetos a partir de múltiplas linhas tabulares com redundâncias.

NoSQL: Documento Agregado

```
{  
  "_id": 1,  
  "nome": "Maria Clara Silva",  
  "endereco": {"cidade": "Taubá", "uf": "CE"},  
  "pedidos": [  
    {  
      "id_pedido": "PED-001",  
      "itens": [  
        {"produto": "Notebook", "qtd": 1}  
      ]  
    }  
  ]  
}
```

Leitura e escrita direta em 1 único I/O atômico de disco.

Matriz Comparativa: Relacional vs NoSQL

Dimensão	Relacional (RDBMS)	Não-Relacional (NoSQL)
Estrutura	Esquema rígido e normalizado	Esquema dinâmico e flexível (schemaless)
Escalabilidade	Predominantemente Vertical (Scale-Up)	Horizontal nativa em cluster (Scale-Out)
Consistência	Imediata e estrita (ACID)	Eventual ajustável (BASE / CAP)
Relacionamentos	Otimizado para JOINS locais	Desnormalização ou Grafos dedicados
Cenário Ideal	Sistemas bancários, ERPs, sistemas financeiros	E-commerce, telemetria IoT, Big Data, IA / RAG

Próximos Passos & Atividades

Próximo Encontro: 06/10/2026

Aula 02 – Teorema CAP e BASE

- O Teorema CAP de Eric Brewer detalhado.
- Teorema PACELC: a troca entre consistência e latência.
- Comparação formal: ACID vs BASE.

Tarefas do Estudante

- Executar o script didático de apoio:
`codigo/demonstracao_aula01.py`.
- Iniciar os exercícios do Bloco 1 da **Lista de Exercícios 1 (AV3)**.
- Instalar o Docker para a oficina do primeiro sábado letivo (10/10).

Dúvidas & Discussão

IFCE Campus Tauá – ADS16

reginaldo.fernandes@ifce.edu.br